This is a much safer check between the two variables; always use strict comparison when dealing with user input. Input validation is also essential: if we'd tested the user-supplied token's data type to make sure it was a string, we would have been alerted to problems before comparison. Input validation is something we will explore in Chapter 11, "Programming for the Internet."

Comparison Coercion

The security issue demonstrated above was triggered by a loosely typed comparison, and historically, this has been a source of bugs and confusion. PHP will coerce strings on both sides of a comparison to numbers if they are numeric:

```
var_dump('0e1' == '0e2');
var_dump('0e1' === '0e2');
var_dump('0e1' == '0');
var_dump('0e1' === '0');
```

```
Output:
bool(true)
bool(false)
bool(true)
bool(false)
```

We're comparing two scientific notation strings (which represent the floats *0.0* and *0.00*) and a scientific notation string to an integer string (*0.0* and *0*). You can see that in both cases a loosely typed comparison evaluates to `true`, but the strongly typed comparison is `false`.

Naturally, if one of the operands is a number type, the string will be coerced. However, a new feature of PHP 8 is that it will only coerce numeric strings in these cases; numerical strings (containing non-digit characters) are not due to the historic issues around numerical coercion. If you wish to loosely compare a numerical string to a number, it is now necessary to explicitly cast it first.

```
var_dump('100' == 100);
var_dump('100 monkeys' == 100);
var_dump((int) '100 monkeys' == 100);
```